

Sport Dashboard API Backend – Projektplan

Projektziel

Ziel ist die Entwicklung eines eigenständigen Backend-Systems, das sämtliche Sportdaten zentral verwaltet und über eine REST-API bereitstellt.

Die bestehende Flask-Website (sport-dashboard-web) soll künftig ausschließlich auf diese API zugreifen und keinerlei direkte Verbindungen zu externen Datenquellen oder Datenbanken mehr besitzen.

Anforderungen

- Zentrale Speicherung aller Sportdaten in einer Datenbank
- Trennung von Frontend und Backend
- Hohe Sicherheit der Datenbank
- Automatische Aktualisierung von Sportdaten
- Historische Datenhaltung
- Lokale Entwicklungsumgebung
- Produktivsystem auf Render
- Möglichst geringe laufende Kosten
- Erweiterbar für weitere Projekte (App, Discord Bot, etc.)

Phase 1 – Architektur aufsetzen

Technologie-Stack

Backend:

- Python 3.13+
- FastAPI
- Uvicorn

Datenbank:

- PostgreSQL
- SQLAlchemy
- Alembic

Scheduling:

- APScheduler

HTTP Requests:

- httpx

Crawler:

- BeautifulSoup4
- Playwright (nur falls nötig)

Validierung:

- Pydantic

Phase 2 – Grundstruktur erstellen

Projektstruktur:

sport-dashboard-api/

```
├─ app/
|
├─ api/
|   └─ v1/
|
├─ models/
|
├─ schemas/
|
├─ services/
|
├─ crawlers/
|
├─ schedulers/
|
├─ database/
|
└─ migrations/
```

```
|
|
├── tests/
|
├── .env
|
├── alembic.ini
|
├── requirements.txt
|
└── main.py
```

Phase 3 – Datenbank einrichten

PostgreSQL auswählen

Entwicklung:

- Lokale PostgreSQL Instanz

Produktiv:

- Render PostgreSQL

Erste Datenbankmodelle

Competitions

- id
- name
- sport
- country
- season

Teams

- id
- name
- short_name
- country
- logo_url

Matches

- id
- competition_id
- home_team_id
- away_team_id
- kickoff_time
- status
- home_score
- away_score
- last_updated

Drivers

- id
- name
- team
- number

Races

- id
- season
- name
- date
- status

NFL Teams

- id
- name
- conference
- division

Phase 4 – Datenbankmigrationen

Alembic integrieren

Ziel:

Schemaänderungen automatisch verwalten.

Workflow:

Model ändern

↓

Migration erstellen

↓

Git Push

↓

Deployment

↓

Migration wird automatisch ausgeführt

Beispiel:

```
alembic revision --autogenerate -m "add matches table"
```

```
alembic upgrade head
```

Phase 5 – Lokale Entwicklungsumgebung

Docker verwenden

Dateien:

- docker-compose.yml

Services:

- postgres
- pgadmin (optional)

Ziele:

- Lokale Entwicklung unabhängig von Produktion
- Identische Umgebung
- Einfaches Setup

Start:

- docker compose up

Phase 6 – API-Grundgerüst

API Versionierung

- /api/v1/

Beispiele:

- /api/v1/bundesliga/table
- /api/v1/bundesliga/matches
- /api/v1/f1/races
- /api/v1/nfl/games

OpenAPI / Swagger

FastAPI automatisch nutzen:

- /docs
- /redoc

Phase 7 – Authentifizierung

Rollenmodell

Public

Nur Lesen.

Admin

Darf:

- Daten aktualisieren
- Scheduler starten
- Crawler ausführen

API-Key System

Beispiel Header:

- X-API-Key

Später optional:

- JWT
- OAuth

Phase 8 – Scheduler entwickeln

APScheduler integrieren

Scheduler läuft dauerhaft im Backend.

Wettbewerbskalender importieren

Beispiel:

- Bundesliga 2026/27

Alle Spiele werden gespeichert.

Dynamische Jobs erstellen

Für jedes Spiel:

Kickoff bekannt

↓

Endzeit schätzen

↓

Ab Endzeit:

alle 2 Minuten prüfen

↓

Spiel beendet?

↓

Ja

↓

Daten aktualisieren

↓

Job löschen

Keine Endzeiten hardcoden.

Phase 9 – Datenquellen integrieren

Formel 1

Quelle:

- OpenF1

Daten:

- Kalender
- Ergebnisse
- Fahrer
- Teams

NFL

Quelle:

- Tank01

Daten:

- Spielplan
- Ergebnisse
- Tabellen

Fußball

Priorität 1

Zuerst die Daten über diese API holen

- API-Football

Priorität 2

Crawler entwickeln.

Architektur:

Crawler Interface

└─ KickerCrawler

└─ FlashscoreCrawler

└─ SofascoreCrawler

Jeder liefert dieselbe Datenstruktur zurück.

Phase 10 – Datenimport-Pipeline

Ablauf

Scheduler

↓

Datenquelle

↓

Validierung

↓

Normalisierung

↓

Datenbank

↓

API Endpunkte

Ziel:

Alle Quellen liefern identische Datenmodelle.

Phase 11 – Logging

Logging einführen

Speichern:

- API Requests
- Scheduler Jobs
- Crawler Fehler
- Import Fehler

Log-Level:

- INFO
- WARNING
- ERROR

Phase 12 – Monitoring

Health Endpoint:

- /api/v1/health

Antwort:

```
{  
  "status": "ok"  
}
```

Verwendung:

- UptimeRobot
- Render Health Check

Phase 13 – Vorbereitung darauf das ein Flask Projekt umgestellt wird

Bestehendes Projekt:

sport-dashboard-web

Aktuell:

Flask

↓

Externe APIs

Neu:

Flask

↓

Sport Dashboard API

Die Website kennt künftig:

- keine Datenbank
- keine Scheduler
- keine Crawler
- keine externen APIs

Phase 14 – Sicherheit

Datenbank

- Nicht öffentlich erreichbar
- Nur API Server darf zugreifen
- Eigener Datenbank-User
- Starke Passwörter

Secrets

Nur über:

Environment Variables

niemals im Git Repository.

SQL Sicherheit

Nur SQLAlchemy ORM verwenden.

Keine SQL-Strings zusammenbauen.

Phase 15 – Deployment

Render

Service 1

- sport-dashboard-api

Service 2

- PostgreSQL

Die Crawler sollen im ersten Entwurf noch nicht vorhanden sein erstmal soll die Grund Struktur vorhanden sein und Funktionieren